Семинар 9. Консенсус, RAFT и FLP-теорема

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Через неделю дедлайн по третьему домашнему заданию
- Проверки 2-го и 3-го домашнего задания проставлю чуть позже в таблицу

Содержание

1. Синхронные и асинхронные распределенные системы
2. Понятие консенсуса
3. Алгоритмы консенсуса
4. FLP-теорема
5. Алгоритм консенсуса RAFT
6. Дискуссия

Синхрония в распределенных системах

Синхронные системы

- Известно, что любое сообщение либо доставляется за некоторое время C, либо полностью исчезает
- Тогда можно разбить выполнение алгоритма на фазы длиной O(C): в начале фазы все процессы отправляют сообщения, потом ждут, а в конце знают, что получили все сообщения, которые только могли прийти

Асинхронные системы

- Время передачи сообщения сверху не ограничено
- Но гарантируется, что если нет отказов, то сообщение рано или поздно дойдёт

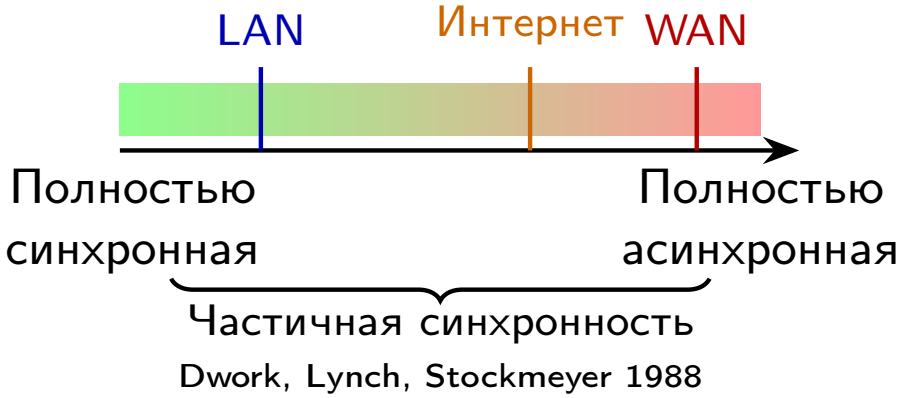
Как узлы воспринимают время?

Синхронная модель

- Задержка сообщений $\leq \delta$ (известно)
- Таймаут = надёжный детектор отказов
- Примеры: real-time системы, авионика самолета

Асинхронная модель

- Задержка сообщений — произвольная
- «Упала нода» ≡ «нода очень медленная»
- Примеры: интернет, облачные кластеры



Реальность ≈ **частичная синхронность**: обычно быстро, но иногда сеть «проваливается»

Что может пойти не так? Модели отказов

Crash-stop нода молча останавливается, больше не отвечает

Crash-recovery упала и восстановилась (с дисковым журналом)

Omission нода работает, но теряет часть сообщений

Byzantine произвольное поведение: ложь, подделка, сговор

сложность алгоритма

Сегодня

Фокус на **crash-stop**. Таймаут — надёжный детектор отказов *только* в синхронной модели.

Содержание

1. Синхронные и асинхронные распределенные системы
2. Понятие консенсуса
3. Алгоритмы консенсуса
4. FLP-теорема
5. Алгоритм консенсуса RAFT
6. Дискуссия

Задача консенсуса

Консенсус

n процессов, каждый **предлагает** значение v_i. Все должны **принять решение** — одно общее значение.

Три свойства:

Validity Если все предложили одно v — решение равно v.
Защита от «тупого» алгоритма, который всегда возвращает 0.

Agreement Все честные процессы принимают *одинаковое* решение.
Суть задачи: никакого раскола.

Termination Каждый честный процесс *в итоге* принимает решение.
Алгоритм не зависает навечно.

Заметка

Иногда добавляют **Integrity**: решение обязано быть значением, которое предложил хотя бы один процесс (не «из воздуха»).

Где консенсус встречается в жизни?

Distributed commit (2PC)

Все узлы БД должны решить: **COMMIT** или **ABORT**.
Если координатор — единственная точка отказа, 2PC не даёт консенсуса при сбое координатора.

State Machine Replication

Реплики должны применять команды в **одном порядке**.
Иначе состояния разойдутся.

Leader election

Кто координатор прямо сейчас?
Два «лидера» одновременно — катастрофа (split-brain).

Блокчейн

Какой блок следующий в цепочке?
Proof-of-Work, Proof-of-Stake — всё это варианты консенсуса.

Синхронный консенсус: $f+1$ раундов

Алгоритм (crash-stop, синхронная модель, f возможных отказов):

1. На каждом раунде $r = 1, \ldots, f+1$: каждый живой процесс рассылает *всем* все значения, которые он узнал к этому моменту.
2. После $f+1$ раундов: взять min из всех известных значений (или любую детерминированную функцию).

Почему $f+1$ раундов достаточно?

В худшем случае каждый раунд «забирает» одну ноду ровно перед отправкой. За f раундов могут упасть f нод — но их данные *уже успели разойтись*. На раунде $f+1$ все живые узлы имеют одинаковую картину.

Стоимость: $O(n^2)$ сообщений за раунд $\Rightarrow O(f \cdot n^2)$ всего.
Дорого при больших f, но *корректно*.

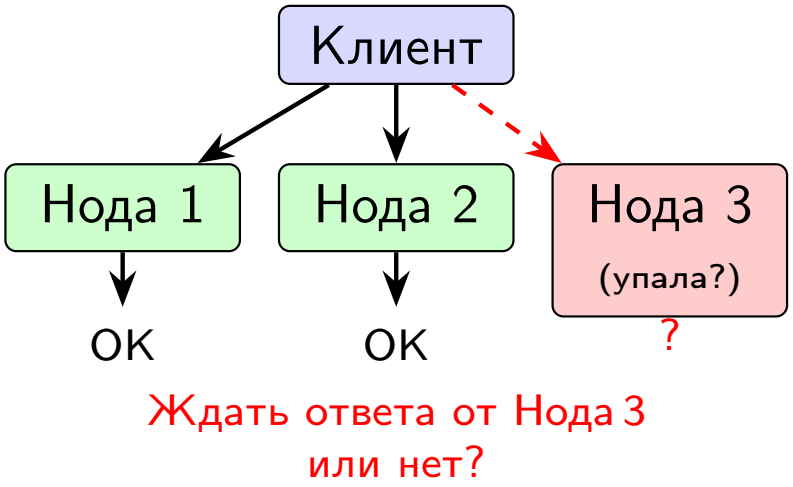
Содержание

1. Синхронные и асинхронные распределенные системы
2. Понятие консенсуса
3. Алгоритмы консенсуса
4. FLP-теорема
5. Алгоритм консенсуса RAFT
6. Дискуссия

Почему асинхронный консенсус сложен?

Дилемма алгоритма:

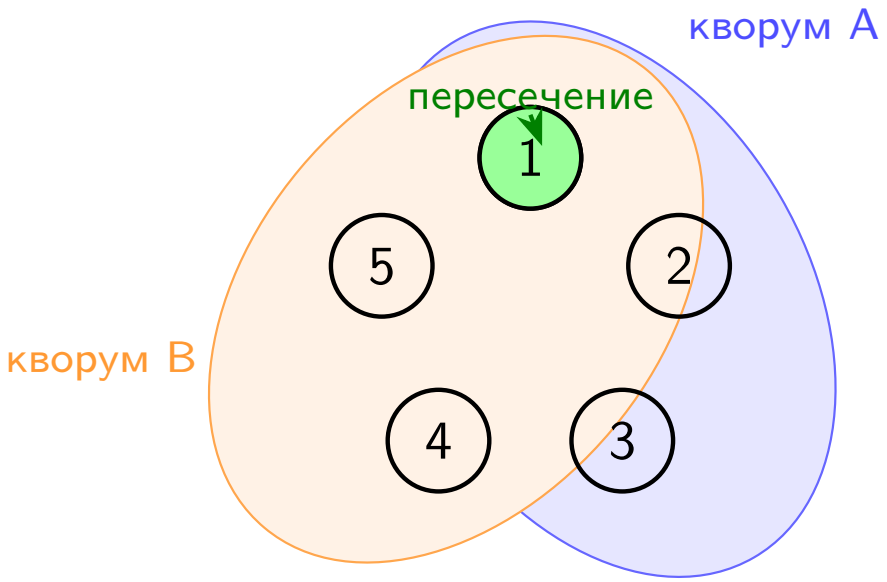
- **Ждать все ответы?**
  Если нода упала — зависнем навечно.
  Нарушение Termination.
- **Не ждать всех?**
  Разные процессы соберут разные наборы голосов и могут принять *разные* решения.
  Нарушение Agreement.



Кворумы: большинство как инструмент

Почему $\lfloor n/2 \rfloor + 1$?

Любые два кворума **пересекаются** хотя бы в одном узле. Этот узел «помнит» предыдущее решение — не даёт двум лидерам принять разные значения.



Аналогия

У вас 5 копий ключа от сейфа. Чтобы открыть — нужно 3. Любые две группы по 3 человека *обязательно* пересекаются.

Работает при $f < n/2$ crash-stop отказов.

Содержание

1. Синхронные и асинхронные распределенные системы
2. Понятие консенсуса
3. Алгоритмы консенсуса
4. FLP-теорема
5. Алгоритм консенсуса RAFT
6. Дискуссия

FLP: невозможность консенсуса

Теорема (Fischer, Lynch, Paterson, 1985)

В **полностью асинхронной** системе, где хотя бы **один** процесс может упасть (crash-stop), не существует *детерминированного* алгоритма консенсуса, который *всегда завершается*.

Иными словами: **Safety** (Agreement) и **Liveness** (Termination) — нельзя гарантировать одновременно в асинхронной модели при наличии отказов.

FLP на практике: приговор или руководство?

FLP говорит об *асинхронной* модели с *детерминированным* алгоритмом. Оба ограничения можно ослабить:

1. **Частичная синхронность** (GST-модель, Dwork–Lynch–Stockmeyer)
   После некоторого момента *GST* сеть ведёт себя синхронно.
   Paxos и Raft завершаются *в периоды стабильности*.
2. **Рандомизация** (Ben-Or 1983)
   «Монетка» делает алгоритм не детерминированным.
   Консенсус достигается с вероятностью 1 — но не в худшем случае.
3. **Failure Detectors** (Chandra–Toueg 1996)
   Добавить оракул, который *иногда неточно* детектирует отказы.
   Класс Ω (eventually perfect leader detector) достаточен для консенсуса.

Содержание

1. Синхронные и асинхронные распределенные системы
2. Понятие консенсуса
3. Алгоритмы консенсуса
4. FLP-теорема
5. Алгоритм консенсуса RAFT
6. Дискуссия

Демо

Презентация – Мартин Клеппманн
https://thesecretlivesofdata.com/raft/

Содержание

1. Синхронные и асинхронные распределенные системы
2. Понятие консенсуса
3. Алгоритмы консенсуса
4. FLP-теорема
5. Алгоритм консенсуса RAFT
6. Дискуссия

Итоги

- **Модели систем:** синхронная / асинхронная / частично-синхронная; иерархия отказов от crash-stop до Byzantine.
- **Консенсус** — три свойства: Validity, Agreement, Termination. В синхронной модели решается за $f+1$ раундов; в асинхронной — нетривиально.
- **Алгоритмы:** кворумы + лидер — общая идея Paxos, Raft и других. Paxos и Raft подробно — на следующем семинаре.
- **FLP (1985):** в полностью асинхронной системе детерминированный консенсус *невозможен*. Обходы: частичная синхронность, рандомизация, failure detectors.